

TASK 1

To Create Process Escape Sequences, Handle Escaped Spaces, Escape Special Symbols, Preserve Characters, Validate Parser Output, Test Complex Inputs

```
#include <stdio.h>
```

```
#include <string.h>
```

```
#define MAX 1000
```

```
/*
```

```
    ESCAPE SEQUENCE PARSER
```

```
    Features:
```

1. Process escape sequences
2. Handle escaped spaces
3. Escape special symbols
4. Preserve escaped characters
5. Validate parser output
6. Test complex inputs

```
*/
```

```
void parse_escape_sequences(const char *input, char *output)
```

```
{
```

```
    int i = 0;
```

```
    int j = 0;
```

```
    while (input[i] != '\0')
```

```
    {
```

```
        /*
```

If a backslash is found,
process the next character as escaped.

```
*/
```

```
if (input[i] == '\\')
```

```
{
```

```
    i++;
```

```
    if (input[i] == '\0')
```

```
    {
```

```
        /*
```

Backslash at the end of the input.

```
    */
```

```
    output[j++] = '\\';
```

```
    break;
```

```
}
```

```
/*
```

Preserve the escaped character.

This handles:

\ space

\\$

\&

\;

\\

\'

\"

\\

```
*/
```

```
output[j++] = input[i];
```

```

    }
    else
    {
        output[j++] = input[i];
    }

    i++;
}

output[j] = '\0';
}

/*
    Check whether the parser preserved
    the expected output.
*/
void test_case(const char *input, const char *expected)
{
    char output[MAX];

    parse_escape_sequences(input, output);

    printf("\n-----\n");
    printf("Input   : %s\n", input);
    printf("Expected : %s\n", expected);
    printf("Output  : %s\n", output);

    if (strcmp(output, expected) == 0)

```

```
        printf("Result  : PASS\n");
    else
        printf("Result  : FAIL\n");
}
```

```
int main()
{
    printf("=====\n");
    printf("    ESCAPE SEQUENCE PARSER\n");
    printf("=====\n");

    /*
        Test 1: Escaped space
    */
    test_case(
        "Hello\\ World",
        "Hello World"
    );

    /*
        Test 2: Escaped dollar symbol
    */
    test_case(
        "Price\\$100",
        "Price$100"
    );

    /*
```

Test 3: Escaped ampersand

*/

```
test_case(  
    "Hello\\&World",  
    "Hello&World"  
);
```

/*

Test 4: Escaped semicolon

*/

```
test_case(  
    "Hello\\;World",  
    "Hello;World"  
);
```

/*

Test 5: Escaped pipe

*/

```
test_case(  
    "Hello\\|World",  
    "Hello|World"  
);
```

/*

Test 6: Escaped single quote

*/

```
test_case(  
    "It\\'s",  
    "It's"
```

```
);
```

```
/*
```

```
    Test 7: Escaped double quote
```

```
*/
```

```
test_case(
```

```
    "Say\\ \"Hello\\\"",
```

```
    "Say \"Hello\\\"",
```

```
);
```

```
/*
```

```
    Test 8: Escaped backslash
```

```
*/
```

```
test_case(
```

```
    "C:\\\\Users\\\\Korsa",
```

```
    "C:\\Users\\Korsa"
```

```
);
```

```
/*
```

```
    Test 9: Multiple escaped spaces
```

```
*/
```

```
test_case(
```

```
    "This\\ is\\ a\\ test",
```

```
    "This is a test"
```

```
);
```

```
/*
```

```
    Test 10: Complex input
```

```
*/
```

```
test_case(  
    "Hello\\ World\\ \\$USER\\ \\&\\ Test",  
    "Hello World $USER & Test"  
);  
  
printf("\n=====\\n");  
printf("All escape sequence tests completed.\\n");  
printf("=====\\n");  
  
return 0;  
}
```

```

nikith@NIKITH:~$ mkdir -p ~/ossp
cd ~/ossp
nikith@NIKITH:~/ossp$ nano escape_parser.c
nikith@NIKITH:~/ossp$ gcc escape_parser.c -o escape_parser
nikith@NIKITH:~/ossp$ ./escape_parser
=====
          ESCAPE SEQUENCE PARSER
=====

-----
Input   : Hello\ World
Expected: Hello World
Output  : Hello World
Result  : PASS

-----
Input   : Price$100
Expected: Price$100
Output  : Price$100
Result  : PASS

-----
Input   : Hello&World
Expected: Hello&World
Output  : Hello&World
Result  : PASS

-----
Input   : Hello;World
Expected: Hello;World
Output  : Hello;World
Result  : PASS

-----
Input   : Hello|World
Expected: Hello|World
Output  : Hello|World
Result  : PASS

-----
Input   : It\'s
Expected: It's
Output  : It's
Result  : PASS

-----
Input   : Say\ "Hello"
Expected: Say "Hello"
Output  : Say "Hello"
Result  : PASS

-----
Input   : C:\\Users\\Korsa
Expected: C:\Users\Korsa
Output  : C:\Users\Korsa
Result  : PASS

-----
Input   : This\ is\ a\ test
Expected: This is a test
Output  : This is a test
Result  : PASS

-----
Input   : Hello\ World\ \ $USER\ \&\ Test
Expected: Hello World $USER & Test
Output  : Hello World $USER & Test
Result  : PASS

=====
All escape sequence tests completed.
=====
nikith@NIKITH:~/ossp$

```

TASK 2

To Create Child Processes, Execute Programs, Handle Execution Errors, Pass Arguments, Manage Parent Process, Test Command Launching

#include <stdio.h>

#include <stdlib.h>

#include <unistd.h>

```

#include <sys/types.h>
#include <sys/wait.h>
#include <errno.h>
#include <string.h>

int main(int argc, char *argv[])
{
    if (argc < 2) {
        fprintf(stderr, "Usage: %s <command> [arguments...]\n", argv[0]);
        return 1;
    }

    printf("Parent PID: %d\n", getpid());

    pid_t pid = fork();           // 1. Create child process

    if (pid < 0) {                // fork failed
        perror("fork failed");
        return 1;
    }

    else if (pid == 0) {          // ---- CHILD ----
        printf("Child PID: %d (Parent: %d)\n", getpid(), getppid());
        printf("Launching: %s\n\n", argv[1]);

        execvp(argv[1], &argv[1]); // 2 & 4. Execute program, pass arguments

        // Only reached if exec fails // 3. Handle execution errors
        fprintf(stderr, "exec failed for '%s': %s\n", argv[1], strerror(errno));
        exit(127);
    }
}

```

```

}

else {                                // ---- PARENT ----

    int status;

    waitpid(pid, &status, 0);    // 5. Manage parent: wait for child

    if (WIFEXITED(status)) {

        int code = WEXITSTATUS(status);

        printf("\nChild exited with status %d\n", code);

        if (code == 127)

            printf("Command could not be launched.\n");

    } else if (WIFSIGNALED(status)) {

        printf("\nChild killed by signal %d\n", WTERMSIG(status));

    }

    printf("Parent (PID %d) finished.\n", getpid());

}

return 0;

}

```

```

nikith@NIKITH:~$ nano launcher.c
nikith@NIKITH:~$ gcc launcher.c -o launcher
nikith@NIKITH:~$ ./launcher ls -l          # runs ls with an argument
./launcher echo Hello World                # passes multiple arguments
./launcher date                             # runs date
./launcher pwd
./launcher wrongcmd                         # tests error handling (exec fails, status 127)
Parent PID: 731
Child PID: 732 (Parent: 731)
Launching: ls

total 344
-rw-r--r-- 1 nikith nikith 80 Aug 24 05:15 File1.txt
-rw-r--r-- 1 nikith nikith 80 Aug 24 05:22 File2.txt
prw-r--r-- 1 nikith nikith 0 Aug 29 08:29 NIKITH
drwxr-xr-x 10 nikith nikith 4096 Aug 24 05:12 OSSP
-rwxr-xr-x 1 nikith nikith 16480 Aug 17 05:16 command_executor
-rw-r--r-- 1 nikith nikith 1105 Aug 17 05:14 command_executor.c
-rwxr-xr-x 1 nikith nikith 16352 Sep 21 05:19 double_quotes
-rw-r--r-- 1 nikith nikith 3228 Sep 21 05:19 double_quotes.c
-rwxr-xr-x 1 nikith nikith 16216 Aug 24 05:22 filecopy
-rw-r--r-- 1 nikith nikith 1806 Aug 24 05:22 filecopy.c
-rwxr-xr-x 1 nikith nikith 16088 Aug 24 05:06 fork_example
-rw-r--r-- 1 nikith nikith 466 Aug 24 05:06 fork_example.c
-rwxr-xr-x 1 nikith nikith 16168 Aug 24 05:07 fork_exec
-rw-r--r-- 1 nikith nikith 630 Aug 24 05:07 fork_exec.c
-rwxr-xr-x 1 nikith nikith 15952 Aug 17 05:47 hello
-rw-r--r-- 1 nikith nikith 78 Aug 17 05:47 hello.c
-rwxr-xr-x 1 nikith nikith 16512 Sep 21 05:32 launcher
-rw-r--r-- 1 nikith nikith 1540 Sep 21 05:32 launcher.c
-rwxr-xr-x 1 nikith nikith 16384 Aug 24 05:08 my_shell
-rw-r--r-- 1 nikith nikith 13 Aug 29 08:13 myfile.txt
-rwxr-xr-x 1 nikith nikith 16992 Aug 24 05:36 myshell
-rw-r--r-- 1 nikith nikith 7461 Aug 24 05:36 myshell.c
drwxr-xr-x 2 nikith nikith 4096 Sep 21 05:25 ossp
-rwxr-xr-x 1 nikith nikith 16136 Aug 17 05:50 process_states
-rw-r--r-- 1 nikith nikith 433 Aug 17 05:50 process_states.c
-rw-r--r-- 1 nikith nikith 678 Aug 24 05:08 shell.c
-rw-r--r-- 1 nikith nikith 2891 Aug 17 05:06 task1.c
-rwxr-xr-x 1 nikith nikith 16248 Aug 17 05:32 task3
-rw-r--r-- 1 nikith nikith 1101 Aug 17 05:32 task3.c
-rwxr-xr-x 1 nikith nikith 16248 Aug 17 05:39 task4
-rw-r--r-- 1 nikith nikith 1302 Aug 17 05:38 task4.c
-rwxr-xr-x 1 nikith nikith 16392 Aug 24 05:26 week4_task1
-rw-r--r-- 1 nikith nikith 1976 Aug 24 05:25 week4_task1.c
-rwxr-xr-x 1 nikith nikith 16208 Aug 24 05:27 zombie
-rw-r--r-- 1 nikith nikith 759 Aug 24 05:27 zombie.c
-rwxr-xr-x 1 nikith nikith 16264 Aug 24 05:29 zombie_fixed
-rw-r--r-- 1 nikith nikith 1099 Aug 24 05:29 zombie_fixed.c

Child exited with status 0
Parent (PID 731) finished.
Parent PID: 738
Child PID: 739 (Parent: 738)
Launching: echo

Hello World

Child exited with status 0
Parent (PID 738) finished.
Parent PID: 745
Child PID: 746 (Parent: 745)
Launching: date

Mon Sep 21 05:33:05 UTC 2026

Child exited with status 0
Parent (PID 745) finished.
Parent PID: 752
Child PID: 753 (Parent: 752)
Launching: pwd

/home/nikith

Child exited with status 0
Parent (PID 752) finished.
Parent PID: 759
Child PID: 760 (Parent: 759)
Launching: wrongcmd

exec failed for 'wrongcmd': No such file or directory

Child exited with status 127
Command could not be launched.
Parent (PID 759) finished.
nikith@NIKITH:~$

```